



Guía del Estudiante — API REST con Docker y MySQL

Asignatura DevOps | Actividad práctica

DUOC UC · DevOps 003D

¿Qué vas a lograr? Levantar una API REST de Estudiantes con base de datos MySQL en tu propio computador, usando contenedores. No necesitas instalar Java, Maven ni MySQL — solo la herramienta de contenedores de tu preferencia.



Paso 0 — Elige tu herramienta

Antes de comenzar, identifica cuál tienes instalada:

Herramienta	¿Cómo verificar?	¿Gratis?
Docker Desktop	<code>docker --version</code>	✓ Uso personal
Podman	<code>podman --version</code>	✓ 100% open source

Abre una terminal y prueba ambos comandos. Usa el que responda. A lo largo de esta guía verás bloques alternativos para cada uno — elige solo el tuyo.



Paso 1 — Obtén el proyecto

Descomprime el archivo `mysql-local.zip` que te entregó el profesor.

```
mysql-local/
├─ Dockerfile           ← receta para construir el contenedor de la app
├─ docker-compose.yml   ← orquesta la app + MySQL juntos
├─ pom.xml              ← dependencias del proyecto Java
└─ src/                 ← código fuente de la aplicación
```

Abre una terminal y entra a la carpeta:

```
cd mysql-local
```



Todos los comandos del resto de esta guía se ejecutan dentro de `mysql-local/`.



Paso 2 — Prepara tu herramienta

Si usas Podman

Paso 2a — Inicia la máquina virtual de Podman

```
podman machine start
```

Espera este mensaje antes de continuar:

```
Machine "podman-machine-default" started successfully
```

Paso 2b — Instala podman-compose (si no lo tienes)

```
pip3 install podman-compose
```

Verifica:

```
podman-compose --version
```

Si usas Docker Desktop

Asegúrate de que Docker Desktop esté **abierto y corriendo** (ícono en la barra de menú).

Verifica en la terminal:

```
docker --version
docker compose version
```

Ambos comandos deben responder sin error.

🔍 Paso 3 — Lee el Dockerfile antes de construir

Abre el archivo `Dockerfile` con cualquier editor de texto y observa su estructura.

```
# ETAPA 1: Build (el taller)
FROM eclipse-temurin:17-jdk AS build # imagen con Java 17 completo + compilador

RUN apt-get install -y maven          # instala Maven (herramienta de compilacion)

COPY pom.xml .                        # copia la lista de dependencias
RUN mvn dependency:go-offline          # descarga todas las librerías

COPY src ./src                        # copia el código fuente
RUN mvn clean package -DskipTests     # compila, genera el archivo .jar

# ETAPA 2: Runtime (el producto final)
FROM eclipse-temurin:17-jre          # imagen liviana, solo para ejecutar

COPY --from=build /app/target/bdget-0.0.1-SNAPSHOT.jar app.jar # copia solo el .jar

EXPOSE 8080                           # puerto de la aplicación
ENTRYPOINT ["java", "-jar", "app.jar"] # comando de arranque
```

¿Por qué dos etapas?

La Etapa 1 tiene Maven, el JDK completo y el código fuente (~700 MB). La Etapa 2 solo tiene el `.jar` final + el JRE (~280 MB). La imagen que se despliega es **solo la Etapa 2** — más liviana y más segura.

🔍 Paso 4 — Lee el docker-compose.yml antes de ejecutar

Abre el archivo `docker-compose.yml` y ubica estos puntos clave:

```

services:

  mysql:                                # Contenedor 1: base de datos
    image: mysql:8.0                    # imagen oficial, se descarga de internet
    environment:
      MYSQL_DATABASE: bdget_db          # crea la BD automaticamente al arrancar
      MYSQL_USER: bdget_user
      MYSQL_PASSWORD: bdget_pass
    healthcheck:                         # verifica que MySQL este realmente listo
      test: ["CMD-SHELL", "mysqladmin ping ..."]
    volumes:
      - mysql_data:/var/lib/mysql       # los datos se guardan aunque el contenedor se detenga

  app:                                  # Contenedor 2: la aplicacion Spring Boot
    build: .                             # construye la imagen usando el Dockerfile
    ports:
      - "8080:8080"                     # tu PC:8080 -> contenedor:8080
    environment:
      - spring.datasource.url=jdbc:mysql://mysql:3306/bdget_db # "mysql" = nombre del servicio
    depends_on:
      mysql:
        condition: service_healthy      # espera que MySQL este listo primero

```

💡 **Punto importante:** La URL de la base de datos dice `mysql:3306` (no `localhost:3306`). Dentro de la red de compose, los contenedores se llaman por el **nombre del servicio**. `localhost` dentro del contenedor `app` es el propio contenedor, no MySQL.

🚀 Paso 5 — Levanta el sistema completo

Este es el comando principal. Construye las imágenes y arranca ambos contenedores:

Con Podman

```
podman-compose up --build
```

Con Docker

```
docker compose up --build
```

¿Qué ocurre al ejecutarlo?

Observa la salida en la terminal — verás 4 fases:

Fase 1 — Descarga de imágenes base

```

Pulling mysql:8.0 ...
Pulling eclipse-temurin:17-jdk ...

```

Fase 2 — Build de la aplicación (Etapa 1 del Dockerfile)

```

Step: RUN apt-get install -y maven
Step: RUN mvn dependency:go-offline    ← puede tardar 2-3 min la primera vez
Step: RUN mvn clean package

```

Fase 3 — MySQL arranca y pasa el healthcheck

```

[mysql] MySQL init process done. Ready for start up.
[mysql] ready for connections. Port: 3306

```

Fase 4 — La app arranca y se conecta a MySQL

```
[app] Started BdgetApplication in X.XXX seconds
```

🕒 **La primera vez demora ~3-5 minutos** porque descarga dependencias de internet. Las siguientes veces tarda ~30 segundos gracias al caché.

✅ Paso 6 — Verifica que todo está corriendo

Abre **otra terminal** (deja la primera con los logs) y ejecuta:

Con Podman

```
podman ps
```

Con Docker

```
docker ps
```

Debes ver **dos contenedores** en estado Up:

NAMES	IMAGE	STATUS	PORTS
bdget-app	mysql-local_app	Up	0.0.0.0:8080->8080/tcp
bdget-mysql	mysql:8.0	Up	0.0.0.0:3306->3306/tcp

🌐 Paso 7 — Abre la interfaz Swagger

La aplicación incluye **Swagger UI**: una interfaz web para probar todos los endpoints sin necesidad de comandos ni herramientas adicionales.

Abre tu navegador en:

```
http://localhost:8080/swagger-ui.html
```

Verás una página con todos los endpoints disponibles:

GET	/students	→ Lista todos los estudiantes
POST	/students	→ Crea un nuevo estudiante
GET	/students/{id}	→ Busca un estudiante por ID
PUT	/students/{id}	→ Actualiza un estudiante
DELETE	/students/{id}	→ Elimina un estudiante

🔧 Paso 8 — Prueba la API

Puedes probar la API de **dos maneras** — elige la que prefieras:

Opción A — Desde Swagger UI (navegador)

1. Abre `http://localhost:8080/swagger-ui.html`
2. Click en POST /students → click en "Try it out"
3. En el campo Request body escribe:

```
{
  "name": "Ana"
}
```

4. Click en **"Execute"**

5. Observa la respuesta `201 Created`:

```
{
  "id": 1,
  "name": "Ana"
}
```

Opción B — Desde la terminal con curl

Listar todos (lista vacía al inicio)

```
curl http://localhost:8080/students
```

Respuesta: `[]`

Crear un estudiante

```
curl -X POST http://localhost:8080/students \
-H "Content-Type: application/json" \
-d '{"name": "Ana"}
```

Respuesta esperada: `{"id": 1, "name": "Ana"}`

Crear otro estudiante

```
curl -X POST http://localhost:8080/students \
-H "Content-Type: application/json" \
-d '{"name": "Luis"}
```

Listar todos

```
curl http://localhost:8080/students
```

Respuesta esperada: `[{"id": 1, "name": "Ana"}, {"id": 2, "name": "Luis"}]`

Buscar por ID

```
curl http://localhost:8080/students/1
```

Actualizar

```
curl -X PUT http://localhost:8080/students/1 \
-H "Content-Type: application/json" \
-d '{"name": "Maria"}
```

Eliminar

```
curl -X DELETE http://localhost:8080/students/2
```

Prueba la validación (debe fallar con error 400)

La aplicación valida que el nombre: no esté vacío, tenga entre 2 y 50 caracteres, y solo contenga letras (sin números ni símbolos).

```
# Nombre con números → error
curl -X POST http://localhost:8080/students \
  -H "Content-Type: application/json" \
  -d '{"name": "Ana123"}'
```

Respuesta esperada (**400 Bad Request**):

```
{
  "status": 400,
  "errors": {
    "name": "El nombre solo puede contener letras"
  }
}
```

```
# Nombre vacío → error
curl -X POST http://localhost:8080/students \
  -H "Content-Type: application/json" \
  -d '{"name": ""}'
```

Respuesta esperada:

```
{
  "status": 400,
  "errors": {
    "name": "No puede ingresar un nombre vacío"
  }
}
```

Paso 9 — Verifica los datos en MySQL

Entra al contenedor de MySQL y revisa los datos directamente en la base de datos:

Con Podman

```
podman exec -it bdget-mysql mysql -u bdget_user -pbdget_pass bdget_db
```

Con Docker

```
docker exec -it bdget-mysql mysql -u bdget_user -pbdget_pass bdget_db
```

Una vez dentro del cliente MySQL, ejecuta:

```
-- Ver las tablas que creó Spring Boot automáticamente
SHOW TABLES;
```

```
+-----+
| Tables_in_bdget_db |
+-----+
| student             |
+-----+
```

```
-- Ver la estructura de la tabla
DESCRIBE student;
```

```
+-----+-----+-----+-----+
| Field | Type          | Null | Key |
+-----+-----+-----+-----+
| id    | bigint        | NO   | PRI |
| name  | varchar(50)   | NO   |     |
+-----+-----+-----+-----+
```

```
-- Ver los estudiantes creados
SELECT * FROM student;
```

```
+----+-----+
| id | name |
+----+-----+
| 1  | Maria |
+----+-----+
```

```
-- Salir de MySQL
EXIT;
```

💡 Observa que la tabla `student` fue creada **automáticamente** por Spring Boot al arrancar, gracias a `spring.jpa.hibernate.ddl-auto=update` en `application.properties`. No escribiste ningún SQL para crear la tabla.

🔴 Paso 10 — Detener y reiniciar

Detener sin borrar nada

Presiona `Ctrl + C` en la terminal con los logs, luego:

Con *Podman*

```
podman-compose stop
```

Con *Docker*

```
docker compose stop
```

Los contenedores quedan detenidos. Los datos de MySQL se conservan en el volumen.

Volver a levantar (sin reconstruir)

Con *Podman*

```
podman-compose start
```

Con *Docker*

```
docker compose start
```

✅ Los estudiantes que creaste antes seguirán ahí — el volumen los preservó.

Reiniciar desde cero (borra todos los datos)

Con *Podman*

```
podman-compose down -v
```

Con Docker

```
docker compose down -v
```

La bandera `-v` elimina el volumen de MySQL. La próxima vez que levantes con `up --build` la base de datos estará vacía.

Resumen de comandos

Acción	Podman	Docker
Levantar todo (primera vez)	<code>podman-compose up --build</code>	<code>docker compose up --build</code>
Levantar (sin reconstruir)	<code>podman-compose up</code>	<code>docker compose up</code>
Ver contenedores activos	<code>podman ps</code>	<code>docker ps</code>
Ver logs en vivo	<code>podman-compose logs -f</code>	<code>docker compose logs -f</code>
Detener (conserva datos)	<code>podman-compose stop</code>	<code>docker compose stop</code>
Reanudar	<code>podman-compose start</code>	<code>docker compose start</code>
Eliminar contenedores	<code>podman-compose down</code>	<code>docker compose down</code>
Eliminar contenedores + datos	<code>podman-compose down -v</code>	<code>docker compose down -v</code>
Entrar a MySQL	<code>podman exec -it bdget-mysql mysql -u bdget_user -pbdget_pass bdget_db</code>	<code>docker exec -it bdget-mysql mysql -u bdget_user -pbdget_pass bdget_db</code>
Entrar a la app	<code>podman exec -it bdget-app /bin/sh</code>	<code>docker exec -it bdget-app /bin/sh</code>

Solución de problemas frecuentes

"address already in use" en el puerto 8080 o 3306

Otro proceso está usando ese puerto. Identifícalo y deténlo:

```
# Busca qué proceso usa el puerto 8080
lsof -i :8080

# Busca qué proceso usa el puerto 3306
lsof -i :3306
```

La app arranca pero no se conecta a MySQL

El healthcheck de MySQL no pasó a tiempo. Espera 30 segundos y vuelve a intentar:

```
podman-compose down
podman-compose up
```

"podman machine" no está corriendo (solo Podman)

```
podman machine start
```

El build falla con error de Maven

Verifica que tienes conexión a internet — Maven necesita descargar dependencias la primera vez. Luego reintenta:

```
podman-compose up --build
# o
docker compose up --build
```

✅ Lista de verificación — entregables

Al terminar la actividad debes tener evidencia de cada punto:

- ☐ Captura de `podman ps` / `docker ps` con los dos contenedores en estado `Up`
- ☐ Captura del navegador en `http://localhost:8080/swagger-ui.html`
- ☐ Captura de `GET /students` con al menos 2 estudiantes creados
- ☐ Captura del error `400` al intentar crear un estudiante con nombre inválido
- ☐ Captura del resultado de `SELECT * FROM student` dentro del contenedor MySQL